Forward Kinematics Design

Forward Kinematics Design

1. Course Content
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

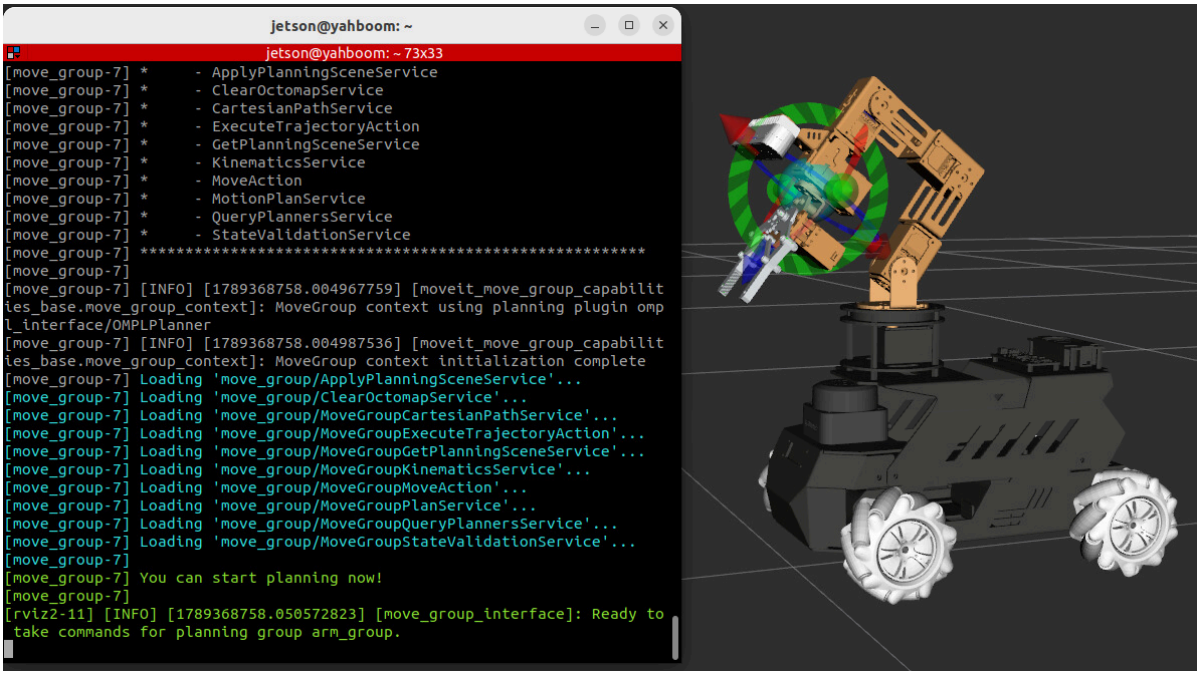
1. Course Content

This course explains how to call the functions in the MoveIt library to implement forward kinematics design of the robot arm. Forward kinematics design refers to computing the pose of the robot arm's end effector based on the joint angles. Here, in RViz, we specify the angle of each joint, and then let MoveIt2 plan and move to this pose.

2. Preparation

```
ros2 launch yahboomcar_bringup moveit_simulation.launch.py sim2real:=true
```

Once the program starts, when **You can start planning now!** appears in the terminal, it means the startup was successful, as shown in the figure below:



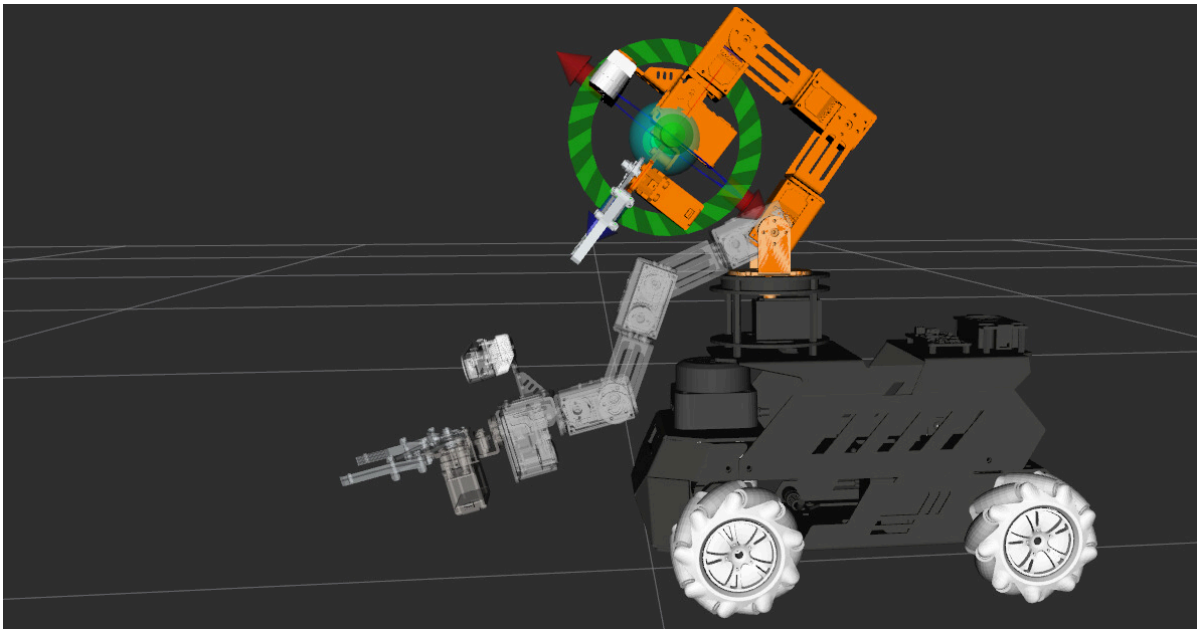
3. Case Demonstration

- Enter the following command in the terminal to launch the forward kinematics design program:

```
ros2 run moveit_demo set_target_joints --ros-args -p target_joints:="
[90.0,-24.0,44.0,146.0,90.0]"
```

> **Note**
>
> - target_joints is the angle of robot arm joints 1-5, as floating-point numbers. The above command is a reference example; the angles can be specified by yourself through parameters.

After the program runs, the robot arm in RViz will plan and move to the pose of each joint we set, as shown in the figure below:



```
jetson@yahboom:~$ ros2 run moveit_demo set_target_joints --ros-args -p target_joints:="[90
.0,-24.0,44.0,146.0,90.0]"
[INFO] [1789371895.128233428] [set_target_joints]: Initializing SetTargetJoints.
[INFO] [1789371898.197412503] [moveit_rdf_loader.rdf_loader]: Loaded robot model in 3.0679
8 seconds
[INFO] [1789371898.197558526] [moveit_robot_model.robot_model]: Loading robot model 'ROSMA
STER_M1Pro'...
[INFO] [1789371898.197579039] [moveit_robot_model.robot_model]: No root/virtual joint spec
ified in SRDF. Assuming fixed joint
[WARN] [1789371900.221128013] [moveit_ros.robot_model_loader]: No kinematics plugins defin
ed. Fill and load kinematics.yaml!
[INFO] [1789371900.257042746] [move_group_interface]: Ready to take commands for planning
group arm_group.
[INFO] [1789371900.257120509] [set_target_joints]: Target joints (deg): [90.00, -24.00, 44
.00, 146.00, 90.00]
[INFO] [1789371900.257180864] [set_target_joints]: Target joints (rad): [1.5708, -0.4189,
0.7679, 2.5482, 1.5708]
[INFO] [1789371900.257397258] [move_group_interface]: MoveGroup action client/server ready
[INFO] [1789371900.260706018] [move_group_interface]: Planning request accepted
[INFO] [1789371900.308982501] [move_group_interface]: Planning request complete!
[INFO] [1789371900.309913552] [move_group_interface]: time taken to generate plan: 0.02903
46 seconds
```

4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/moveit_demo/src/set_target_joints.cpp
```

initialize function description:

- `declare_parameter<std::vector<double>>("target_joints", ...)`: Declares the target joint angle parameter, with a default value of `[86.0, 47.0, -26.0, 89.0, 90.0]`, which can be overridden via the `-p` command line.
- `declare_parameter<bool>("ignore_obstacles", false)`: Declares the obstacle ignore switch, off by default.
- `target_joints_deg.size() != 5`: Verifies that the number of joint angles must be 5; if it does not match, an error is reported and the node is shut down.
- `deg * M_PI / 180.0`: Converts the angle values into radians one by one; MoveIt2 internally uses radians for motion planning.
- `MoveGroupInterface(shared_from_this(), "arm_group")`: Creates a MoveGroup control interface, bound to the `arm_group` planning group.
- `setNumPlanningAttempts(10)`: Sets the number of planning attempts per call to 10.
- `setPlanningTime(5.0)`: Sets the maximum time for a single planning attempt to 5 seconds.
- `setMaxVelocityScalingFactor(1.0)` / `setMaxAccelerationScalingFactor(1.0)`: Both velocity and acceleration scaling factors are 1.0, using the full speed of the planned motion.
- `clearSceneObstacles()`: Called when `ignore_obstacles:=true`, clears all collision objects and the octomap in the scene.
- `setStartStateToCurrentState()`: Sets the planning start state to the robot arm's current state.
- `setJointValueTarget(target_joints_rad)`: Sets the target joint angles (radians), i.e., the input to forward kinematics.
- `plan(my_plan)`: Plans a trajectory for the target joint angles.
- `execute(my_plan)`: If planning succeeds, executes the trajectory and drives the robot arm to the target pose.
- `printEndEffectorPose(target_joints_rad)`: After execution, computes and prints the pose of the end effector in the measure coordinate frame.

clearSceneObstacles function description:

- `PlanningSceneInterface`: Creates a planning scene interface, used to manage the collision objects in the scene.
- `getKnownObjectNames()`: Gets the list of names of all known collision objects in the scene.
- `removeCollisionObjects(object_ids)`: Removes all collision objects in the scene in batches.
- `create_publisher<PlanningScene>("/planning_scene", 10)`: Creates a planning scene publisher, used to send scene update messages.
- `ps_msg.is_diff = true`: Marks it as an incremental update message, modifying only part of the scene content.
- `ps_msg.world.octomap.header.frame_id = "measure"`: Sets the reference frame of the octomap; publishing an octomap with empty data indicates clearing the existing map data.
- `sleep_for(300ms)` / `sleep_for(500ms)`: Waits for the message publication to take effect, ensuring the scene update is complete.

printEndEffectorPose function description:

- `getRobotModel()`: Gets the robot model, which contains information about all joints and links.
- `RobotState(robot_model)`: Creates a robot state object, used for kinematics computation.
- `getJointModelGroup("arm_group")`: Gets the joint model group corresponding to the arm_group planning group.
- `setJointGroupPositions(jmg, joint_positions)`: Sets the given joint angles into the robot state.
- `robot_state.update()`: Updates the robot state, triggering the forward kinematics solution.
- `getGlobalLinkTransform("gripper_tcp")`: Gets the pose transformation matrix of the gripper end link in the global coordinate frame.
- `getGlobalLinkTransform("measure")`: Gets the pose transformation matrix of the measure-frame link in the global coordinate frame.
- `measure_pose.inverse() * gripper_tcp_pose`: Transforms the gripper pose into the measure coordinate frame to obtain the relative pose.
- `std::atan2(rot(2,1), rot(2,2))` / `std::asin(-rot(2,0))` / `std::atan2(rot(1,0), rot(0,0))`: Extracts the RPY Euler angles from the rotation matrix (ROS fixed-axis XYZ convention) and converts them to degrees.

```
class SetTargetJoints : public rclcpp::Node
{
public:
  SetTargetJoints()
    : Node("set_target_joints")
  {
    RCLCPP_INFO(this->get_logger(), "Initializing SetTargetJoints.");
  }

  void initialize()
  {
    // Declare ROS parameters: target joint angles in degrees and obstacle ignore
flag
    this->declare_parameter<std::vector<double>>(
      "target_joints",
      std::vector<double>{86.0, 47.0, -26.0, 89.0, 90.0});
    this->declare_parameter<bool>("ignore_obstacles", false);

    bool ignore_obstacles = this->get_parameter("ignore_obstacles").as_bool();
    std::vector<double> target_joints_deg = this-
>get_parameter("target_joints").as_double_array();

    if (target_joints_deg.size() != 5)
    {
      RCLCPP_ERROR(this->get_logger(),
        "target_joints must have exactly 5 elements, got %zu.",
target_joints_deg.size());
      rclcpp::shutdown();
      return;
    }

    // Convert degrees to radians
```

```
    std::vector<double> target_joints_rad;
    target_joints_rad.reserve(5);
    for (double deg : target_joints_deg)
    {
      target_joints_rad.push_back(deg * M_PI / 180.0);
    }

    // Initialize MoveGroupInterface after shared_ptr is fully constructed
    move_group_interface_ =
std::make_shared<moveit::planning_interface::MoveGroupInterface>(
      shared_from_this(), "arm_group");

    move_group_interface_->setNumPlanningAttempts(10);
    move_group_interface_->setPlanningTime(5.0);
    move_group_interface_->setMaxVelocityScalingFactor(1.0);
    move_group_interface_->setMaxAccelerationScalingFactor(1.0);

    // Optionally clear all scene obstacles (collision objects + octomap)
    // so that the target object is not treated as an obstacle during planning
    if (ignore_obstacles)
    {
      RCLCPP_INFO(this->get_logger(), "ignore_obstacles:=true, clearing scene
obstacles...");
      clearSceneObstacles();
    }

    RCLCPP_INFO(this->get_logger(),
      "Target joints (deg): [%.2f, %.2f, %.2f, %.2f, %.2f]",
      target_joints_deg[0], target_joints_deg[1], target_joints_deg[2],
      target_joints_deg[3], target_joints_deg[4]);
    RCLCPP_INFO(this->get_logger(),
      "Target joints (rad): [%.4f, %.4f, %.4f, %.4f, %.4f]",
      target_joints_rad[0], target_joints_rad[1], target_joints_rad[2],
      target_joints_rad[3], target_joints_rad[4]);

    move_group_interface_->setStartStateToCurrentState();
    move_group_interface_->setJointValueTarget(target_joints_rad);
    moveit::planning_interface::MoveGroupInterface::Plan my_plan;
    if (move_group_interface_->plan(my_plan) ==
moveit::core::MoveItErrorCode::SUCCESS)
    {
      RCLCPP_INFO(this->get_logger(), "Planning succeeded, executing.");
      move_group_interface_->execute(my_plan);
      printEndEffectorPose(target_joints_rad);
    }
    else
    {
      RCLCPP_ERROR(this->get_logger(), "Planning failed!");
    }
  }

private:
  void clearSceneObstacles()
  {
    // 1. Remove all known collision objects via PlanningSceneInterface
    planning_scene_interface_ =
```

```
      std::make_shared<moveit::planning_interface::PlanningSceneInterface>();

    std::vector<std::string> object_ids = planning_scene_interface_-
>getKnownObjectNames();
    if (!object_ids.empty())
    {
      RCLCPP_INFO(this->get_logger(),
        "Removing %zu collision objects from scene.", object_ids.size());
      planning_scene_interface_->removeCollisionObjects(object_ids);
    }

    // 2. Clear the octomap by publishing a PlanningScene diff message
    //    Setting frame_id with empty octomap.data signals the
PlanningSceneMonitor
    //    to remove the existing octomap
    auto scene_pub = this->create_publisher<moveit_msgs::msg::PlanningScene>(
      "/planning_scene", 10);
    rclcpp::sleep_for(std::chrono::milliseconds(300));

    moveit_msgs::msg::PlanningScene ps_msg;
    ps_msg.is_diff = true;
    ps_msg.world.octomap.header.frame_id = "measure";
    ps_msg.world.octomap.header.stamp = this->now();

    scene_pub->publish(ps_msg);
    rclcpp::sleep_for(std::chrono::milliseconds(500));

    RCLCPP_INFO(this->get_logger(),
      "Scene obstacles cleared (collision objects removed, octomap cleared).");
  }

  void printEndEffectorPose(const std::vector<double>& joint_positions)
  {

    moveit::core::RobotModelConstPtr robot_model = move_group_interface_-
>getRobotModel();
    moveit::core::RobotState robot_state(robot_model);
    const moveit::core::JointModelGroup* jmg = robot_model-
>getJointModelGroup("arm_group");

    robot_state.setJointGroupPositions(jmg, joint_positions);
    robot_state.update();

    // Get link transforms in the global (model root) frame
    const Eigen::Isometry3d& gripper_tcp_pose =
      robot_state.getGlobalLinkTransform("gripper_tcp");
    const Eigen::Isometry3d& measure_pose =
      robot_state.getGlobalLinkTransform("measure");

    // Compute gripper_tcp pose relative to measure frame
    Eigen::Isometry3d gripper_in_measure = measure_pose.inverse() *
gripper_tcp_pose;

    double x = gripper_in_measure.translation().x();
    double y = gripper_in_measure.translation().y();
    double z = gripper_in_measure.translation().z();
```

```
    // Extract RPY from rotation matrix (ROS convention: fixed-axis XYZ)
    Eigen::Matrix3d rot = gripper_in_measure.rotation();
    double roll = std::atan2(rot(2, 1), rot(2, 2)) * 180.0 / M_PI;
    double pitch = std::asin(-rot(2, 0)) * 180.0 / M_PI;
    double yaw = std::atan2(rot(1, 0), rot(0, 0)) * 180.0 / M_PI;

    RCLCPP_INFO(this->get_logger(),
      "End-effector pose in measure frame: "
      "x=%.4f, y=%.4f, z=%.4f, roll=%.4f deg, pitch=%.4f deg, yaw=%.4f deg",
      x, y, z, roll, pitch, yaw);
  }

  std::shared_ptr<moveit::planning_interface::MoveGroupInterface>
move_group_interface_;
  std::shared_ptr<moveit::planning_interface::PlanningSceneInterface>
planning_scene_interface_;
};
```